

EduVideo: Class and XML Guide

A source-based reference for AI-assisted video screenplays

Jenga-Code / George Miller

4 October 2026

Contents

1	How to use this guide	3
2	Quick start: a complete lesson	3
3	The rules an AI must follow	3
3.1	Structure, positioning and timing	4
3.2	Supported common properties	4
3.3	Themes and backgrounds	5
4	Visual object vocabulary	5
4.1	Text, cards and basic graphics	5
4.2	Geometry and mathematical diagrams	6
4.3	Arrays, matrices, code and graphs	7
5	Animation and screenplay instructions	7
5.1	Ordinary events: AnimationParser	7
5.2	Production events: ProductionEvents	8
5.3	Narration limitations in this source snapshot	9
6	Worked XML recipes	9
6.1	Plot a curve and mark a known point	9
6.2	Show a row-by-column matrix multiplication	10
6.3	Display an algorithm without pretending the XML executes it	10
6.4	Rearrange equation tokens	10
6.5	Assemble chapters into one production	10
7	A reusable instruction for an AI	11
8	Checking a generated lesson	11
8.1	Frequent problems and their actual causes	12
9	Class catalogue	12
9.1	Package <code>algorithm</code> (7 files)	12
9.2	Package <code>animation</code> (14 files)	14
9.3	Package <code>app</code> (4 files)	18
9.4	Package <code>audio</code> (11 files)	19
9.5	Package <code>cli</code> (4 files)	22
9.6	Package <code>editor</code> (1 files)	23
9.7	Package <code>export</code> (3 files)	23
9.8	Package <code>math</code> (16 files)	24
9.9	Package <code>objects</code> (11 files)	28
9.10	Package <code>preview</code> (1 files)	30
9.11	Package <code>production</code> (8 files)	31
9.12	Package <code>project</code> (2 files)	33
9.13	Package <code>render</code> (9 files)	33
9.14	Package <code>scene</code> (7 files)	35
9.15	Package <code>ui</code> (10 files)	37
9.16	Package <code>util</code> (4 files)	40
9.17	Package <code>video</code> (6 files)	41
9.18	Package <code>xml</code> (11 files)	42
10	Source coverage and verification	45

1 How to use this guide

This is a practical reference to the **129 Java source files** in George Miller's uploaded `src.zip`, inspected on 4 October 2026. It documents this source snapshot, not a presumed newer release. The aim is to help you give an AI enough accurate information to write EduVideo XML screenplays.

Read the XML authoring chapters first. The class catalogue then explains each source file, its main functions and its XML connection. Interfaces, records and enums are included because they are part of the program's API. Nested helper types are covered with their owning file rather than treated as separate source files.

The central rule: a Java class or method does not automatically become an XML tag or attribute. Only the vocabulary registered by `ObjectParser`, `StandardObjects`, `VisualObjects`, `AnimationParser` and `ProductionEvents` is available to a normal lesson. `ProductionParser` handles the separate multipart manifest.

The guide was checked against parser branches, model validation and relevant rendering/audio implementations. Examples have been checked for XML structure and registered vocabulary. They have not been executed by the Java application in this environment: the upload contains source/resources but no Maven build descriptor, and a Java compiler is unavailable here. Render a still and a short preview in your own project before a full production.

2 Quick start: a complete lesson

A lesson has an `edu-video` root, an optional theme, and one or more scenes. A scene contains visual objects and sibling event instructions. Times are in seconds from the start of that scene. Scenes play in file order.

```
<?xml version="1.0" encoding="UTF-8"?>
<edu-video version="1.0" width="1920" height="1080" fps="60">
  <theme name="midnight-blue"/>
  <scene id="intro" duration="10" theme="midnight-blue">
    <text id="title" x="96" y="120" size="50" bold="true"
      text="Solving a simple equation"/>
    <equation id="working" x="240" y="420" size="64"
      value="2x + 6 = 14"/>
    <result id="answer" x="620" y="680" width="680"
      height="140" text="x = 4" style="success"/>
    <effect target="title" name="fadeIn" start="0" duration="1"/>
    <typewriter target="working" start="1" duration="2"/>
    <effect target="answer" name="popIn" start="6" duration="1"/>
  </scene>
</edu-video>
```

This example needs no external assets or speech service. The answer remains hidden before its entrance because the first opacity track holds its initial zero value before its start. The equation reveals progressively. To add narration, place a cue directly inside the scene:

```
<speak id="explanation" start="0.5" max-end="5.5"
  language="en-GB" voice="default" rate="1"
  text="Subtract six, then divide both sides by two."/>
```

`max-end` is a deadline, not a command to stretch or shorten generated speech. If the measured clip exceeds its window, production fails. Leave generous time and adjust after measuring the narration.

3 The rules an AI must follow

3.1 Structure, positioning and timing

- Use `version="1.0"`. Root lesson attributes are only `version`, `width`, `height`, `fps`; defaults are 1920 by 1080 at 60 fps. There is no root `title` or `duration` attribute.
- Every scene needs `id` and `duration`. Additional scene attributes: `background`, `theme`, `transition`, `transitionDuration`. Scene duration must be positive.
- IDs must be nonblank and unique across the whole scene, including nested groups. Do not use `camera` for a visual object: it identifies the built-in camera. Separate scenes can reuse object IDs.
- All events belong directly under a scene. Put objects inside containers, but keep their animations as scene-level siblings targeting the nested object's ID. An event is not a child of the object it animates.
- Pixel coordinates normally start at the top left: `x` increases right and `y` increases down. Text `y` is its baseline; circles and graph nodes are centred on their local origin. A rectangle starts at its top left. Lines/arrows run from `(x,y)` to `(x+width,y+height)` in local coordinates.
- Child positions are relative to their container. A panel adds its padding to the children's origin. A step card needs about 40 pixels reserved below that padded origin for its heading. Panels do not automatically clip or lay out their children.
- `function`, `plot-point`, `area` and `calculus` must be **direct children of axes**. Their mathematical values map through the axes range; do not wrap them in an intermediate group.
- Event `start` and `duration` use scene-local seconds; root `audio` and production-manifest audio use their respective global clocks. Keep every event inside its scene.
- Tracks on the same object and same property may meet at their endpoints but must not overlap. Different properties can animate together. Effects can use more than one property.
- Ordinary property tracks hold their first value before the first keyframe and their last value after the end. This can override an object's base attribute even before an animation begins. Use an initial holding keyframe when necessary.
- `visible` is a fixed boolean, not an animated property. For timed appearance use `opacity`, `fadeIn`, `fadeOut` or `popIn`.
- Higher `z` objects render later within their parent. Equal `z` preserves insertion order; `z` does not flatten all nested groups into one global order.

3.2 Supported common properties

The parser recognises these exact spellings, but each class still decides which properties it supports and actually draws. A property accepted by a base class is not a guarantee that it changes every visual.

Property	Meaning / initial value
<code>x</code> , <code>y</code>	Local translation; 0, 0.
<code>width</code> , <code>height</code>	Usually dimensions; base defaults 100, 100. Constructors can override them.
<code>rotation</code>	Signed endpoints for lines/arrows.
<code>scaleX</code> , <code>scaleY</code>	Clockwise degrees; 0.
<code>opacity</code>	Scale about local origin; 1, 1.
<code>color</code>	Alpha, drawn clamped to 0–1; 1. Multiplies parent opacity.
<code>strokeWidth</code>	Explicit colour, normally inherited from the theme foreground by the parser.
<code>drawProgress</code>	Stroke thickness; 3.
<code>glow</code>	Outline/path reveal fraction; 1. Useful for shape/path/plot drawing.
<code>reveal</code>	Illumination intensity; 0. Only meaningful where the renderer uses it.
<code>highlight</code>	Text reveal or triangle angle visibility; 1. Not a general-purpose property.
<code>angleHighlight</code>	Object-specific emphasis; 0. Supported by nodes, edges and triangles; array/code highlights use dedicated events.
<code>zoom</code>	Triangle angle selection: 0 none, 1–3 selected angle.
<code>a</code> , <code>b</code> , <code>c</code>	Camera-only positive magnification; 1.
	Function parameters: 1, 0, 0. <code>a</code> has other meanings for vectors, statistics and calculus.

Other common object attributes are `id`, `z` and `visible`. `dashed="true"` is only allowed on `ShapeObject` subclasses, including primitive shapes, geometry and arrows; it is not universal. Width/height must be nonnegative except

where a class explicitly treats them as signed endpoints.

Colours use `#RRGGBB` or `#RRGGBBAA` (alpha last). Do not use CSS names, `rgb(...)` or three-digit shorthand. Booleans must be `true` or `false`. Attribute names are case-sensitive.

3.3 Themes and backgrounds

Named themes: `dark`, `midnight-blue`, `emerald-green`, `deep-purple`. A root theme can override `background`, `foreground`, `primary`, `secondary`, `success`, `warning` and `font`. A scene theme selects a named palette and creates a procedural backdrop. A root theme alone does not install that scene backdrop.

```
<background style="midnight-blue" grid="true" curves="true"
            dots="true" vignette="true" intensity="0.5"/>
```

This is a scene child, not an object: it has no ID, common transforms or children. Optional `src` loads a background image relative to the lesson XML. `style` selects a named theme. All four toggles default to `true`; intensity defaults to 0.5.

Incoming transitions are `cut` (default), `crossfade`, `fade` and `slide`. Non-cut transitions require a positive `transitionDuration` no longer than the new scene. They operate within the new scene's duration; do not subtract transition time from total lesson duration.

4 Visual object vocabulary

Every row below inherits supported common attributes. The list after each tag is the complete tag-specific attribute vocabulary. An asterisk marks a required attribute in addition to `id`. Defaults and restrictions that matter to authoring follow the list.

4.1 Text, cards and basic graphics

XML tag / Java class	Specific attributes and capabilities
<code>text</code> / <code>TextObject</code>	<code>text</code> , <code>bold</code> , <code>font</code> , <code>size</code> , <code>align</code> . Text can instead be element content. Defaults: size 40, left alignment, not bold, theme font. Alignment: left, center, right.
<code>equation</code> / <code>EquationObject</code>	<code>value</code> , <code>typeset</code> , <code>font</code> , <code>size</code> , <code>align</code> . Content is the fallback value; size 44. <code>typeset=true</code> enables the small maths layout engine. No <code>bold</code> attribute here.
<code>rectangle</code> / <code>RectangleObject</code>	<code>fill</code> (true). Top-left origin.
<code>circle</code> , <code>ellipse</code> / <code>CircleObject</code>	<code>fill</code> (true). Centred origin; width/height are diameters.
<code>point</code> / <code>CircleObject</code>	No extras. Initially 8 by 8 pixels. This is a pixel-space point, distinct from <code>plot-point</code> .
<code>line</code> , <code>segment</code> / <code>LineObject</code>	No extras. Signed width/height give endpoint displacement.
<code>arrow</code> , <code>vector</code> / <code>ArrowObject</code>	No extras. A progressively revealed straight arrow. <code>vector</code> is an alias, not the mathematical <code>vectors</code> diagram.
<code>curved-arrow</code> / <code>CurvedArrowObject</code>	<code>controlX</code> (50), <code>controlY</code> (0), <code>openHead</code> (false). Quadratic control point plus signed width/height endpoint.
<code>image</code> / <code>ImageObject</code>	<code>src*</code> . Local image decoded once; initial size comes from the image. Set dimensions explicitly to scale it.
<code>group</code> / <code>GroupObject</code>	No extras. Nested objects move/scale/fade together.
<code>panel</code> / <code>PanelObject</code>	<code>padding</code> , <code>radius</code> , <code>fillColor</code> , <code>bottomColor</code> , <code>borderColor</code> , <code>borderWidth</code> , <code>shadow</code> . Defaults: padding 24, radius 22, border 1.3, shadow true. Children use padded coordinates.
<code>step</code> / <code>StepObject</code>	All panel extras plus <code>number</code> (1), <code>title*</code> . A numbered instruction card with child content.
<code>result</code> / <code>ResultObject</code>	<code>text</code> , <code>style</code> , <code>size</code> . Content fallback; size 36; style <code>answer</code> , <code>success</code> , <code>warning</code> or <code>info</code> . No nested children.

4.2 Geometry and mathematical diagrams

XML tag / Java class	Specific attributes and capabilities
<code>axes</code> / <code>CoordinatePlane</code>	<code>xmin</code> , <code>xmax</code> , <code>ymin</code> , <code>ymax</code> , <code>grid</code> , <code>step</code> , <code>xlabel</code> , <code>ylabel</code> , <code>gridOpacity</code> , <code>tickSize</code> , <code>axisColor</code> . Defaults: -10..10, -5..5, grid true, step 1, labels x/y, grid opacity 1, tick size 15. Set pixel width/height.
<code>function</code> / <code>FunctionPlot</code>	<code>expression*</code> . Direct axes child; may animate <code>a</code> , <code>b</code> , <code>c</code> and <code>drawProgress</code> .
<code>plot-point</code> / <code>PlotPoint</code>	<code>px</code> , <code>py</code> , <code>label</code> , <code>guides</code> , <code>labelDx</code> , <code>labelDy</code> . Mathematical point defaults 0,0; guides false; label offsets 0,-22 pixels. Direct axes child.
<code>area</code> / <code>AreaPlot</code>	<code>expression*</code> , <code>from</code> (0), <code>to</code> (1), <code>rectangles</code> (0). Zero means continuous shading; 1–2000 gives midpoint rectangles. Require from < to. Direct axes child; not a symbolic integral.
<code>calculus</code> / <code>CalculusObject</code>	<code>expression*</code> , <code>mode</code> (secant), <code>point</code> (1), <code>from</code> (0), <code>to</code> (4). Modes: secant, riemann. Direct axes child. Animate <code>a</code> as secant displacement or Riemann rectangle count.
<code>number-line</code> / <code>NumberLine</code>	<code>min</code> (-5), <code>max</code> (5), <code>step</code> (1). Horizontal real-number axis; width controls length.
<code>polygon</code> / <code>GeometryObject</code>	<code>points*</code> , <code>fill</code> (false). Closed polygon.
<code>path</code> / <code>GeometryObject</code>	<code>points*</code> , <code>closed</code> (false), <code>fill</code> (false). A polyline, optionally closed.
<code>ray</code> / <code>GeometryObject</code>	<code>angle</code> (0). Long ray represented by a finite line. Angle is in degrees.
<code>arc</code> , <code>angle</code> / <code>GeometryObject</code>	<code>radius</code> (60), <code>startAngle</code> (0), <code>extent</code> (90 for arc, 60 for angle). Positive radius; degree angles. <code>angle</code> draws arms and an arc.
<code>triangle</code> / <code>TriangleObject</code>	<code>points*</code> , <code>vertices</code> (A,B,C), <code>sides</code> (three empty labels), <code>sideMask</code> (0), <code>angleMask</code> (0). Exactly three non-collinear points. Masks use bits 1,2,4; sum bits to select multiple items.
<code>vectors</code> / <code>VectorDiagram</code>	<code>mode</code> , <code>ax</code> , <code>ay</code> , <code>bx</code> , <code>by</code> , <code>m00</code> , <code>m01</code> , <code>m10</code> , <code>m11</code> , <code>unit</code> . Modes: addition, projection, linear, complex. Defaults: A=(2,1), B=(1,2), identity matrix, unit 70. Animate <code>a</code> from 0 to 1 to build the construction.
<code>statistics</code> / <code>StatisticsPlot</code>	<code>mode</code> (bars), <code>values*</code> (comma-separated finite numbers). Modes: bars, histogram, scatter, normal. Animate <code>a</code> as included sample count. Scatter plots sample order against value, not arbitrary x,y pairs.
<code>venn</code> / <code>VennDiagram</code>	<code>operation</code> (intersection), <code>left</code> (A), <code>right</code> (B). Operations union, intersection, difference (left minus right); two sets only.
<code>token-equation</code> / <code>TokenEquation</code>	<code>terms*</code> , <code>size</code> (42), <code>spacing</code> (90). Semicolon-separated stable ID=label pairs. <code>transform-equation</code> moves retained token identities into a new arrangement.

Point lists use `x,y x,y x,y`, for example `points="0,0 300,0 150,-220"`. Triangle side/vertex labels use commas. For triangle highlights, `highlight=1..3` selects a side and `angleHighlight=1..3` selects an angle; zero means none. These selections are distinct from the multi-selection masks and zero-based vertex anchors.

The expression grammar supports `x`, `a`, `b`, `c`, `pi`, `e`, arithmetic `+` `-` `*` `/` `^`, parentheses, and single-argument functions `sin`, `cos`, `tan`, `sqrt`, `abs`, `exp`, `log`. Use explicit multiplication: `2*x`, not `2x`. Trigonometric inputs are radians; `log` is natural logarithm. This is a numeric evaluator, not LaTeX or a computer algebra system. Area and calculus evaluations use their fixed function defaults; only `FunctionPlot` exposes all three expression parameters as animation controls.

Equation typesetting supports Unicode text, `\frac{a}{b}`, `\sqrt{x}`, `\{...\}` and `_ {...}`. It is **not full LaTeX**: do not assume environments, arbitrary commands or packages work. Backslashes are literal in XML. Use `&`, `<`; and `"`; where needed, or CDATA for element text.

Statistics report population variance/standard deviation. The displayed 95% mean interval is a normal approximation. The program does not infer experimental meaning or provide exact small-sample inference.

4.3 Arrays, matrices, code and graphs

XML tag / Java class	Specific attributes and capabilities
<code>matrix</code> / <code>MatrixObject</code>	<code>values*</code> , <code>initiallyVisible</code> (true), <code>cellWidth</code> (70), <code>cellHeight</code> (55), <code>size</code> (28). Commas separate cells, semicolons separate rows. Rectangular grid; indices zero-based. Cell metrics determine layout, not generic width/height.
<code>array</code> , <code>list</code> , <code>stack</code> , <code>queue</code> / <code>ArrayObject</code>	<code>values*</code> , <code>vertical</code> . Comma-separated labels. Vertical defaults true only for stack. These are display arrangements, not automatic data-structure simulations.
<code>code</code> / <code>CodeBlockObject</code>	<code>language</code> (java only), <code>title</code> , <code>lineNumbers</code> (true), <code>size</code> (24). Source goes in element text or CDATA. It is displayed, never executed.
<code>variable</code> / <code>VariableObject</code>	<code>name*</code> , <code>value*</code> . Timed assignments change the displayed value.
<code>graph</code> / <code>GraphObject</code>	No extras. Explicitly positioned node/edge children; no automatic graph layout or search.
<code>tree</code> / <code>TreeObject</code>	No extras. Same rendering as graph plus a connected, acyclic undirected-tree check.
<code>node</code> / <code>NodeObject</code>	<code>label</code> (ID), <code>state</code> (default). States: default, selected, visited, active. Initial size 64 by 64.
<code>edge</code> / <code>EdgeObject</code>	<code>from*</code> , <code>to*</code> , <code>weight</code> , <code>label</code> , <code>directed</code> (false). Weight takes precedence over label. Endpoint nodes must be distinct siblings of the edge.

Matrix labels are immutable after creation. There is no XML `set-cell` or `row-operation` command. For elimination, author successive matrix objects and transitions, or generate the screenplay with Java. `MatrixAlgebra` can calculate numerical matrix operations in Java but has no XML object tag.

5 Animation and screenplay instructions

Unless stated otherwise, these are direct scene children. `start` defaults to 0 and `duration` to 1 where those attributes exist. Required parameters are marked with an asterisk. Do not add common object attributes to an event.

5.1 Ordinary events: `AnimationParser`

Instruction	Allowed attributes and use
<code>animate</code>	<code>target*</code> , <code>property*</code> , <code>from*</code> , <code>to*</code> , <code>start</code> , <code>duration</code> , <code>easing</code> . One property track.
<code>animation</code>	<code>target*</code> , <code>property*</code> . Contains at least two <code>keyframe</code> children. Each permits <code>time</code> (0), <code>value*</code> , <code>easing</code> ; times strictly increase. Destination keyframe easing controls the arriving segment.
<code>effect</code>	<code>target*</code> , <code>name*</code> , <code>start</code> , <code>duration</code> , <code>distance</code> . Names below.
<code>typewriter</code>	<code>target*</code> , <code>start</code> , <code>duration</code> , <code>mode</code> (character or word). Text/equation only.
<code>highlight</code>	<code>target*</code> , <code>line</code> , <code>index</code> , <code>start</code> , <code>duration</code> , <code>color</code> . Code needs line; array needs index; other objects need support for the highlight property. Color affects array highlighting, not every target.
<code>select-code</code>	<code>target*</code> , <code>line*</code> , <code>from*</code> , <code>to*</code> , <code>start</code> , <code>duration</code> . Line numbers are 1-based. Columns are zero-based UTF-16 positions; <code>to</code> is exclusive.
<code>swap</code>	<code>target*</code> , <code>a*</code> , <code>b*</code> , <code>start</code> , <code>duration</code> . Swap zero-based array/list/stack/queue slots. Do not overlap swaps.
<code>set</code>	<code>target*</code> , <code>time</code> , <code>value*</code> . Variable assignment at a time, not a generic property setter.
<code>follow-path</code>	<code>target*</code> , <code>points*</code> , <code>start</code> , <code>duration</code> , <code>easing</code> . Piecewise straight path; controls x and y. Does not expose Java's cubic path/orientation features.
<code>highlight-row</code>	<code>target*</code> , <code>row*</code> , <code>column</code> , <code>start</code> , <code>duration</code> , <code>color</code> . Matrix row highlight; column unused.
<code>highlight-column</code>	<code>target*</code> , <code>column*</code> , <code>row</code> , <code>start</code> , <code>duration</code> , <code>color</code> . Matrix column highlight; row unused.

Instruction	Allowed attributes and use
highlight-cell	target*, row*, column*, start, duration, color. Matrix cell highlight.
reveal-cell, hide-cell	target*, row*, column*, start, duration, color. Visibility change persists from start; duration/color are accepted but do not control a fade.

Easing names are exactly `linear`, `easeIn`, `easeOut`, `easeInOut`, `smoothStep`, `bounce`, `elastic`.

Effect name	Properties used
fadeIn, fadeOut	Opacity 0 to 1, or 1 to 0.
drawOn	drawProgress 0 to 1. Use on an object that draws progressively.
popIn	Opacity plus scaleX/scaleY, scaling 0.92 to 1.
slideIn	Opacity plus x; begins at x+distance and slides to configured x. Distance defaults 40.
glowPulse	Glow 0 to 1 to 0.

Do not combine `fadeIn` and `popIn` over the same interval: both write opacity. Likewise a `follow-path` can conflict with a simultaneous x/y animation or slide entrance.

5.2 Production events: ProductionEvents

Instruction	Allowed attributes and use
marker	id*, time (0). Named scene-local time.
speak	id, start, max-end or maxEnd (one required), text, language, voice, rate, pitch, ssml. Plain narration text can be element content.
music, sound	src*, start, end, volume, fade-in, fade-out, loop. End defaults to scene end. Volumes default 0.1 for music and 1 for sound. Music is ducked around narration.
matrix-product-step	left*, right*, result*, row, column, start, duration, work-x, work-y. Matrix IDs and zero-based destination indices; row/column default 0. Work position defaults canvas centre x and 75% height.
transfer	from*, to*, text*, start, duration, bend, size, color. Animated copy of a label between anchors. Defaults: bend -100, size 36, color #4CF0B3. Does not change source/destination data.
transform-equation	target*, terms*, start, duration. Target must be token-equation. Author transforms in chronological order.
move-to	target*, anchor (center), to*, start, duration. Align a source anchor to a destination anchor.
replacement-transform	source*, target*, start, duration. Moves/fades the source into the destination while revealing it; not arbitrary geometric morphing.
assign	target*, value*, start. Variable assignment; unlike <code>set</code> , uses start rather than time.
focus	targets*, start, duration, dim (0.25). Comma-separated IDs; dims unselected top-level objects. Select a containing group when needed.
camera-focus	target*, anchor (center), zoom (1.15), start, duration. Samples a target position at start and pans/zooms to it.
camera-reset	target, anchor, zoom, start, duration. Resets x/y to 0 and zoom to 1; target/anchor/zoom are accepted but ignored.
camera-follow	target*, anchor (center), start, duration. Follows a live object anchor, controlling camera x and y.
camera-frame	x, y, width, height, padding, start, duration. Width/height must be supplied positive; padding defaults 0.1 and must satisfy $0 \leq \text{padding} < 0.5$.

Markers can be referenced as @name or @name+seconds in start, end, time and max-end. The alias maxEnd is accepted for numeric narration deadlines but is not included in marker substitution. duration does not accept marker expressions. Use numeric marker definitions.

```
<marker id="answer-time" time="6"/>
<effect target="answer" name="fadeIn"
  start="@answer-time" duration="0.8"/>
```

Anchor references are different from time markers. They use object@anchor; a bare object ID means its centre.

Object	Anchor forms
General objects	center, left, right, top, bottom, origin, baseline. Origin/baseline both refer to local (0,0); bounds determine the other anchors.
ArrayObject	element:index; zero-based slot anchor, for example items@element:0.
MatrixObject	cell:r:c, row:r, column:c; all zero-based. Example M@cell:0:1.
CoordinatePlane	math:x:y; for example plot@math:2:4.
TriangleObject	vertex:label or zero-based vertex:index; for example tri@vertex:A.
TokenEquation	term:tokenID; for example eq@term:x.

5.3 Narration limitations in this source snapshot

The active implementation is OpenAISpeechProvider. Configure its model, voice and credentials through the application's settings, not invented XML attributes. voice="default" resolves the configured voice. rate must be between 0.25 and 4.0. The provider rejects ssml=true and nonzero pitch even though the generic model/parser can represent those values. Each cue is limited by this implementation to 4096 Unicode code points.

language="en-GB" is stored in the model, but the inspected request sends only model, voice, input, response format and speed. It does not send the language as a separate parameter or accent instruction. Therefore this XML field alone does not guarantee a British accent.

Legacy comments in AudioCue describe silent export. The actual ProjectParser adds those cues to AudioTimeline, and ProductionBuilder mixes audio. FrameExporter/FFmpegExporter are the separate older silent path. Use the production pipeline for narration, music and effects.

6 Worked XML recipes

6.1 Plot a curve and mark a known point

Insert this content into a scene with duration at least 6 seconds. The author supplies the point; the engine does not solve for roots or intersections.

```
<axes id="plot" x="250" y="230" width="1300" height="600"
  xmin="-3" xmax="3" ymin="-1" ymax="9" step="1">
  <area id="shade" expression="x^2" from="0" to="2"
    color="#38BDF8"/>
  <function id="curve" expression="x^2" color="#4CF0B3"/>
  <plot-point id="p" px="2" py="4" label="(2,4)" guides="true"/>
</axes>
<effect target="curve" name="drawOn" start="0" duration="3"/>
<effect target="shade" name="fadeIn" start="3" duration="1"/>
<effect target="p" name="fadeIn" start="4" duration="1"/>
```

For a secant-to-tangent explanation, put this inside the axes, then add its animation directly to the scene. a is a displacement, not the coefficient in the expression here.

```
<!-- Inside axes: -->
<calculus id="secant" expression="x^2" mode="secant" point="1"/>
<!-- Directly inside the scene: -->
<animate target="secant" property="a" from="2" to="0"
```

```
start="1" duration="4" easing="smoothStep"/>
```

6.2 Show a row-by-column matrix multiplication

Insert into a scene of at least 8 seconds. Here $[1,2]$ times $[3;4]$ equals $[11]$. The authored result is checked numerically by `MatrixProductStep`; it is not an empty result placeholder.

```
<matrix id="A" x="220" y="300" values="1,2"
  cellWidth="100" cellHeight="80" size="40"/>
<matrix id="B" x="650" y="260" values="3;4"
  cellWidth="100" cellHeight="80" size="40"/>
<matrix id="C" x="1150" y="300" values="11"
  initiallyVisible="false"
  cellWidth="100" cellHeight="80" size="40"/>
<matrix-product-step left="A" right="B" result="C"
  row="0" column="0" start="1" duration="6"
  work-x="860" work-y="660"/>
```

For a larger product, supply the complete correct result matrix and one event for each output cell. Use separated time windows. For ordinary highlights, use `highlight-row`, `highlight-column` or `highlight-cell` with zero-based indices.

6.3 Display an algorithm without pretending the XML executes it

Insert into a scene of at least 6 seconds. The swap is explicitly authored; showing the Java code does not run it.

```
<code id="code" x="120" y="230" width="900" height="260"
  size="30" title="Swap two entries"><![CDATA[int temp = a[0];
a[0] = a[1];
a[1] = temp;
]]></code>
<array id="items" x="1150" y="300" values="8,3,5"/>
<variable id="temp" x="1150" y="500" name="temp" value="-"/>
<highlight target="code" line="1" start="0" duration="1"/>
<assign target="temp" value="8" start="1"/>
<highlight target="items" index="0" start="1" duration="1"/>
<swap target="items" a="0" b="1" start="2" duration="2"/>
```

If your code text starts with a blank line, remember it may affect line numbering. Prefer putting the first source line immediately after the CDATA opening when precise line references matter.

6.4 Rearrange equation tokens

Insert into a scene of at least 6 seconds. Stable token IDs preserve visual continuity; changed labels are authored explicitly. This demonstrates a rearrangement, not algebra performed by the engine.

```
<token-equation id="eq" x="480" y="440" size="54" spacing="160"
  terms="x=x;plus=+;two=2;equal==;five=5"/>
<transform-equation target="eq" start="2" duration="2"
  terms="x=x;equal==;five=5;minus=-;two=2"/>
```

The label `=` is represented by `equal==`: the first equals sign splits identity from label. Distinct term IDs must be unique within each token list.

6.5 Assemble chapters into one production

This is a **separate XML file**, not a scene or a lesson child. The lesson files must already exist. Relative paths are resolved from the manifest directory.

```
<?xml version="1.0" encoding="UTF-8"?>
<edu-production version="1.0" title="Algebra Chapter 1"
  width="1920" height="1080" fps="30"
```

```

        output="output/algebra.mp4">
    <parts>
      <part src="01-intro.xml" title="Introduction"/>
      <part src="02-equations.xml" title="Solving equations"/>
    </parts>
  </edu-production>

```

Root production attributes are only `version`, `title`, `width`, `height`, `fps`, `output`. Defaults: 1920 by 1080, 30 fps, output/production.mp4. One `parts` element is required; one optional `audio` element may contain `music` children. Production music accepts `src`, `start`, `end`, `volume`, `loop`, `fade-in`, `fade-out`, using the global production clock. Its volume default is 0.15. Individual chapter clocks and IDs remain local to each lesson.

The delivery profile controls actual output dimensions/fps: PREVIEW is 960x540 at 30, STANDARD is 1920x1080 at 30, FINAL is 1920x1080 at 60. The renderer preserves the lesson's logical coordinate system when producing the selected output profile.

7 A reusable instruction for an AI

Copy this instruction together with this guide and your lesson brief. The Markdown companion is especially convenient as AI context; the PDF is for browsing and the LaTeX file for editing/typesetting.

Write a complete EduVideo version 1.0 XML lesson using only the vocabulary in the attached source guide. Treat that guide as the implementation contract. Do not convert arbitrary Java class names or methods into XML tags. State any requested capability that the XML does not support before proposing a supported alternative.

Use 1920x1080 logical coordinates, a midnight-blue theme, readable text and explicit positions. Organise the explanation into scenes, with clear worked steps and room for results. Put objects inside their valid containers and all animation/production events directly under their scene. Keep IDs unique within each scene, including nested objects, and reserve the ID camera.

Calculate all mathematical results independently before writing their display labels. Do not assume the program executes displayed code, solves algebra or calculates a trace from narration. Author all variable changes, swaps, highlights and equation states explicitly. Use only the supported expression and equation grammars.

Plan scene-local timing first. Keep events inside each scene. Do not overlap property tracks, including tracks created by effects. Respect zero-based matrix/array indices and one-based code lines. Place function, plot-point, area and calculus directly under axes. Use the exact supported attributes, values, booleans, colours and anchor formats.

If narration is requested, use short plain-text speak cues with `voice="default"`, `rate="1"`, `language="en-GB"`, and generous numeric max-end deadlines. Do not promise an accent from the language field. Do not use SSML, nonzero pitch, credentials or unsupported voice/model attributes. If no audio assets were supplied, do not invent file paths.

Return the complete XML in one code block, followed by a short scene timing table and a list of any required assets. Check XML escaping, references, attribute spellings, arithmetic, index bounds and timing conflicts. Explain that measured speech timing and final appearance must be checked by the actual program.

8 Checking a generated lesson

1. Save the XML in your existing EduVideo project, beside any relative assets or with correct relative paths.
2. Validate it using the application's parser. Fix the first reported line/element error before judging later errors.
3. Render representative stills: the title, the busiest worked step, and the final answer. Check text baselines, card padding, mathematical ranges and overlap.
4. Render a short preview. Confirm that effects do not conflict and that the pause before a result is long enough.

5. If using speech, let production measure the actual cached/generated clips. Extend scene/cue windows when needed; a guessed reading speed is not validation.
6. Export the final profile to a new filename. The production builder refuses to overwrite an existing output or report.

The `cli.Main` class accepts arguments such as the following. Supply them through your existing IDE/JAR launcher; no build command or JAR filename is assumed because the uploaded archive contains no build descriptor.

```
validate lesson.xml
frame lesson.xml --time 3 output.png --profile preview
build lesson.xml output.mp4 --profile final
build lesson.xml preview.mp4 --profile preview --no-speech
```

`--no-speech` omits narration but keeps music and sound effects. It is not the same as a completely silent render. Full production requires the configured external FFmpeg executable and speech configuration when narration is used.

8.1 Frequent problems and their actual causes

Symptom	Check
Unknown object/attribute	A class or Java method was mistaken for XML syntax, or spelling/case is wrong.
Plot-parent error	Function/area/plot-point/calculus is not directly inside axes.
Duplicate object ID	IDs collide across nested containers or with camera.
Unsupported property	The target class does not support that property; for example generic rectangles do not support text reveal.
Overlapping animation	An effect also controls opacity, x or scale; inspect all generated property tracks.
Object appears too early/late	First-track endpoint holding overrides base properties; visible is not timed.
Matrix explanation error	Wrong dimensions, indices, nonnumeric operand labels or inconsistent authored result.
Narration exceeds window	Actual speech duration is longer than max-end minus start.
Voice options rejected	SSML/nonzero pitch, obsolete voice ID or invalid model configuration.
Text sits outside a card	Text uses a baseline; panels add padding and do not automatically wrap/reflow/clip content.
Export has no narration	The silent exporter was used, no-speech was selected, or there were no speak cues.

9 Class catalogue

The following catalogue covers every Java source file in the archive, grouped by package. Fully qualified names begin `io.jengacode.eduvideo.` followed by the package and class name. File paths are relative to `src/main/java/io/jengacode/eduvideo/`.

Descriptions summarise the supplied implementation and its source documentation. Function lists identify the useful declared entry points; ordinary accessors and constructors are compacted to keep this a guide rather than full Javadoc. Visual classes inherit the positioning, property and animation API of `SceneObject`; container classes also inherit `GroupObject.add` and `children`. A protected `draw/shape` method is an internal rendering hook, not an XML command.

9.1 Package algorithm (7 files)

9.1.1 ArrayObject

Source: `algorithm/ArrayObject.java`

Array, list, queue or stack. Swaps are reconstructed from immutable initial labels. Models array/list/queue/stack labels without mutating initial data during playback. Completed swaps rebuild slot contents; active swaps interpolate positions with a curved lift. Overlapping swaps are rejected.

XML connection: array, list, stack, queue; events highlight and swap.

Main functions:

- `vertical(...)`: Chooses vertical stacking instead of horizontal cells.
- `anchor(...)`: Resolves element:index to the current animated slot centre.
- `size(...)`: Returns the number of stored array slots.
- `swap(...)`: Schedules a non-overlapping exchange between distinct array slots.
- `highlight(...)`: Adds a timed highlight to a zero-based array slot.
- `valuesAt(...)`: Reconstructs labels by applying only swaps completed by the requested time.

9.1.2 CodeBlockObject

Source: `algorithm/CodeBlockObject.java`

Monospaced source code with pluggable syntax tokens and 1-based line highlights. Keeps source immutable and delegates token colours to SyntaxHighlighter. Lines are 1-based; selection columns are zero-based UTF-16 offsets with an exclusive end. Timed line highlights also draw an execution pointer.

XML connection: code; events highlight and select-code.

Main functions:

- `title(...)`: Sets the optional editor filename or heading.
- `lineNumbers(...)`: Shows or hides the code panel line-number gutter.
- `select(...)`: Adds a timed code selection using a 1-based line and an exclusive UTF-16 column end.
- `highlighter(...)`: Replaces the source-preserving syntax token strategy.
- `highlight(...)`: Adds a timed highlight and execution pointer to a 1-based source line.

9.1.3 EdgeObject

Source: `algorithm/EdgeObject.java`

Directed/weighted graph edge, following endpoint animations in its parent graph. Resolves node positions at each scene sample and shortens the line by endpoint radii. Directed heads, labels, draw progress and emphasis follow the same parent coordinate system.

XML connection: edge.

Main functions:

- `from / to(...)`: Return the endpoint node identities used during scene validation and rendering.
- `draw(...)`: Samples both node positions and draws the shortened edge, weight/label and optional arrowhead.

9.1.4 GraphObject

Source: `algorithm/GraphObject.java`

Graph-theory container; coordinates are explicitly supplied for reproducible layouts. Provides a normal transformed group for nodes and edges. Layout is explicit; Scene validation resolves endpoint IDs and checks they share the edge parent.

XML connection: graph.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.1.5 NodeObject

Source: `algorithm/NodeObject.java`

Labeled graph vertex; position is relative to the containing graph. Draws a labeled graph vertex with gradient fill and semantic state colours. Width/height describe an origin-centered ellipse; transient highlight can override the configured state.

XML connection: node; highlight event.

Main functions:

- `bounds(...)`: Returns centred local shape bounds, consistent with the drawing origin.
- `state(...)`: Sets the persistent semantic colour state of this graph vertex.

9.1.6 TreeObject

Source: `algorithm/TreeObject.java`

Graph container requiring one connected, acyclic undirected topology. Checks node/edge membership, n-1 edges and absence of cycles using union-find roots. Together these enforce one connected undirected tree; rendering still uses ordinary graph children.

XML connection: tree.

Main functions:

- `validateTree(...)`: Checks connected acyclic tree topology using node count and union-find roots.

9.1.7 VariableObject

Source: `algorithm/VariableObject.java`

Named value with deterministic step changes. Stores assignments in a sorted map and samples the last assignment at or before the requested time. Backward seeking reconstructs the correct label without undo logic.

XML connection: variable; events set and assign.

Main functions:

- `assign(...)`: Stores a named value change at a nonnegative scene-local time.
- `valueAt(...)`: Returns the latest assigned label at or before the requested time.

9.2 Package animation (14 files)

9.2.1 Animation

Source: `animation/Animation.java`

Pure time-sampled animation contract. Implementations expose their property and scene-local interval and must sample without mutable playback state. This permits rendering frames out of order and scrubbing backward.

Authoring note: Interface implemented by animation tracks; the XML animation tag specifically constructs a `PropertyAnimation`, not an arbitrary implementation.

Main functions:

- `valueAt(...)`: Samples the animation at scene-local time without changing playback state.

9.2.2 CameraFollow

Source: `animation/CameraFollow.java`

Deterministic camera pan track following a live semantic anchor. No previous-frame velocity is stored, so scrubbing and offline rendering produce the same position.

XML connection: camera-follow event.

Main functions:

- `valueAt(...)`: Samples the anchor's world position at a clamped scene time and subtracts the viewport centre.

9.2.3 Easing

Source: `animation/Easing.java`

Named easing curves; implement `DoubleUnaryOperator` for custom curves. Normalizes incoming progress to the closed interval `[0,1]` before evaluating a curve. Elastic easing can overshoot the output interval; callers clamp when required by the property.

Authoring note: Used via the easing attribute; accepted names are listed in the animation chapter. No easing object tag.

Main functions:

- `applyAsDouble(...)`: Evaluates the easing curve after clamping input progress.
- `parse(...)`: Resolves an exact supported XML easing name.

9.2.4 Effects

Source: `animation/Effects.java`

Convenience effects backed exclusively by the existing non-overlapping property timeline. Configures normal property tracks before rendering; it does not create a second clock. Effects on the same property must respect Timeline overlap rules. Pop-in scales around the local origin.

XML connection: effect event.

Main functions:

- `fadeIn(...)`: Animates opacity from zero to one using ease-out timing.
- `fadeOut(...)`: Animates opacity from one to zero using ease-in timing.
- `drawOn(...)`: Reveals the object outline with smooth-step progress.
- `popIn(...)`: Adds opacity and scale entrance tracks around the local origin.
- `slideIn(...)`: Moves horizontally from an offset position while fading in.
- `glowPulse(...)`: Raises and lowers glow intensity using three keyframes.

9.2.5 Keyframe

Source: `animation/Keyframe.java`

Easing controls the segment arriving at this keyframe. A destination keyframe defines the easing of the segment arriving at it. Times must be finite and nonnegative; enclosing tracks require strictly increasing times.

XML connection: keyframe inside animation.

Main functions:

- `Keyframe(...)`: Stores a finite nonnegative time, value and easing curve; the two-argument constructor uses linear easing.

9.2.6 MotionPath

Source: `animation/MotionPath.java`

Pure normalised path sampler. Paths compose with property tracks without simulation state.

Authoring note: Java-only path factory for line, cubic Bezier, circular arc and parametric paths. XML follow-path exposes a polyline only.

Main functions:

- `point(...)`: Samples a position; callers normally supply progress in $[0,1]$.
- `angle(...)`: Returns a tangent angle in radians for optional orientation along the path.
- `line(...)`: Builds a straight path between immutable endpoint copies.
- `bezier(...)`: Builds a cubic Bezier from four copied control points.
- `arc(...)`: Builds a circular arc; angular arguments use radians.
- `parametric(...)`: Adapts user-supplied pure parametric functions.

9.2.7 PathAnimation

Source: `animation/PathAnimation.java`

Arc-length movement along a polyline; pair X and Y tracks on the same path. Copies input points and precomputes cumulative segment lengths. Sampling maps eased progress to distance rather than point count, so unevenly spaced vertices do not change speed unexpectedly.

XML connection: follow-path event.

Main functions:

- `valueAt(...)`: Maps eased elapsed time to cumulative path distance and interpolates its segment.

9.2.8 PathTrack

Source: `animation/PathTrack.java`

One property sampled from a pure path. Three coordinated tracks provide position and optional tangent orientation; scale, opacity and glow remain independent timeline properties.

Authoring note: Java-only wrapper for MotionPath with optional tangent orientation; do not invent an XML orient or bezier attribute.

Main functions:

- `valueAt(...)`: Samples position or tangent orientation in degrees, holding path endpoints outside the interval.
- `follow(...)`: Attaches coordinated position and optional rotation tracks, rejecting property conflicts.

9.2.9 Property

Source: `animation/Property.java`

Properties use doubles; COLOR holds an ARGB integer represented exactly as a double. Defines the XML name and initial value for each numeric property. Objects explicitly declare which properties they support; timeline tracks override initial values.

XML connection: common property attributes and animate/animation property names.

Main functions:

- `parse(...)`: Resolves a property by its exact XML name.

9.2.10 PropertyAnimation

Source: `animation/PropertyAnimation.java`

Immutable keyframe track with endpoint holding. Defensively copies keyframes and interpolates numeric values or individual ARGB channels. Values before the first and after the final keyframe hold at their endpoints.

XML connection: `animate` and `animation` events.

Main functions:

- `valueAt(...)`: Interpolates the enclosing keyframe pair, holding endpoint values outside the track.

9.2.11 PulseAnimation

Source: `animation/PulseAnimation.java`

Discrete highlight pulse; off outside its interval. Uses a half-open `[start,end)` interval. It is suitable for temporary highlights because it returns zero outside that interval instead of holding its final value.

XML connection: `highlight` event for supported objects.

Main functions:

- `valueAt(...)`: Returns one inside `[start,end)` and zero outside.

9.2.12 Timeline

Source: `animation/Timeline.java`

Non-overlapping property tracks, sampled without changing the scene model. Tracks for the same property may touch but cannot overlap. Sampling selects the latest applicable track; ordinary tracks hold endpoints while pulse tracks return zero outside their interval.

Authoring note: Internal property-track container, not a timeline XML element. XML events populate it.

Main functions:

- `trackCount(...)`: Counts property tracks for measured production reports.
- `add(...)`: Adds a track; intervals for the same property may touch but not overlap.
- `value(...)`: Returns the base value or latest applicable track, holding values across gaps.

9.2.13 Transforms

Source: `animation/Transforms.java`

Reusable transform builders compiled into ordinary, conflict-checked property tracks. Matching equations use `TokenEquation`; replacement transforms crossfade object identities.

XML connection: `move-to` and `replacement-transform` events.

Main functions:

- `moveTo(...)`: Moves an object's selected anchor onto a destination, converting scene space to parent space.
- `replace(...)`: Aligns and crossfades source into destination.

9.2.14 ValueTransfer

Source: `animation/ValueTransfer.java`

Transient labelled copy travelling between live anchors. Source objects remain intact. The actor exists only during its interval and can be sampled out of chronological order.

XML connection: `transfer` event.

Main functions:

- `position(...)`: Samples a cubic trajectory between current source and destination locations.

9.3 Package app (4 files)

9.3.1 ApplicationContext

Source: `app/ApplicationContext.java`

Owns background workers for one Studio window and marshals only completion callbacks to JavaFX.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `work(...)`: Runs blocking work outside JavaFX and returns its result on the UI thread.

9.3.2 EduVideoStudio

Source: `app/EduVideoStudio.java`

Primary JavaFX desktop entry point. The CLI remains independently runnable without starting JavaFX.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `main(...)`: Launches the desktop application from Maven or a JavaFX-enabled runtime.

9.3.3 ProjectState

Source: `app/ProjectState.java`

Observable document state. The XML text is authoritative; the parsed model is a validated snapshot.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `observable fields(...)`: Exposes current path, parsed project, dirty flag, status and playhead time to the JavaFX interface.

9.3.4 StudioSettings

Source: `app/StudioSettings.java`

Stores and retrieves Studio/CLI preferences, FFmpeg and cache paths, speech model/voice settings and session or remembered API-key configuration. Can save settings and make a per-job copy.

Authoring note: Application configuration, not screenplay syntax. Settings and credentials do not belong in generated XML.

Main functions:

- `get(...)`: Gets a setting, permitting `-Deduvideo.key` overrides for automation.
- `set(...)`: Changes a preference in memory; save publishes all changes together.
- `save(...)`: Atomically persists preferences without credentials.
- `ffmpeg(...)`: Returns the configured executable, passed as a single process argument.
- `cache(...)`: Returns the shared speech cache folder.
- `speechModel(...)`: Returns the selected supported Speech API model.
- `speechVoice(...)`: Returns the selected voice, using a compatible default when settings predate this release.
- `apiKey(...)`: Returns the configured key, then `OPENAI_API_KEY`.

- `configuredApiKey(...)`: Returns only the GUI-configured key for editing, never copies the environment into the form.
- `keyRemembered(...)`: Reports whether the user explicitly chose local preference storage.
- `setSessionApiKey(...)`: Sets a session-only key for previews, snapshots and tests; does not persist any secret.
- `configureApiKey(...)`: Saves or forgets the key separately from normal settings.
- `copy(...)`: Captures the current session configuration for a render or preview without persisting it.

9.4 Package audio (11 files)

9.4.1 AudioMixer

Source: `audio/AudioMixer.java`

FFmpeg decoder plus deterministic sample-domain mixer. Memory is bounded to one second of output and one decoded stream per track. Music ducking uses measured speech windows. Mixed samples use a soft limiter; final loudness compliance still requires editorial QA.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `preflight(...)`: Checks FFmpeg availability without starting any expensive visual rendering.
- `mix(...)`: Resolves cached narration, verifies its duration and mixes all tracks to a WAV file.
- `run(...)`: Executes FFmpeg with inherited diagnostics and fails on a nonzero status.

9.4.2 AudioSettings

Source: `audio/AudioSettings.java`

Mixing policy: 48 kHz stereo PCM intermediate, AAC delivery and smooth music ducking.

Authoring note: Java mixing policy; not an audio-settings XML element.

Main functions:

- `defaults(...)`: Returns conservative music-under-speech defaults.

9.4.3 AudioTimeline

Source: `audio/AudioTimeline.java`

Audio-only project model; it has no dependency on frame rendering or a wall clock.

XML connection: `speak`, `music` and `sound` compile into this model.

Main functions:

- `speak(...)`: Appends a narration cue; overlap and project bounds are checked during validation.
- `add(...)`: Appends a file-backed music or sound-effect track.
- `narration(...)`: Returns a snapshot of narration requests in authoring order.
- `tracks(...)`: Returns a snapshot of file-backed tracks.
- `validate(...)`: Checks unique cue identities, nonoverlapping speech windows and project bounds.
- `merged(...)`: Merges measured narration ranges so short pauses do not pump music volume.
- `duck(...)`: Samples a smooth duck envelope, returning one outside speech attack/release regions.

9.4.4 NarrationCue

Source: `audio/NarrationCue.java`

Narration window on the global project clock. XML scene-local times are resolved by the parser. `maxEnd` is a strict upper bound, not a request to stretch speech.

XML connection: `speak`.

Main functions: The type supplies the data, contract or internal operations described above; it is not a separate screenplay command.

9.4.5 OpenAISpeechProvider

Source: `audio/OpenAISpeechProvider.java`

The sole production narration provider. Sends text to OpenAI's HTTPS Speech API and requests WAV. Configuration is snapshotted per job; credentials never enter cache keys or diagnostics. No automatic retries: a failed request must not silently create additional billable requests.

Authoring note: Configured outside lesson XML. This implementation rejects SSML and nonzero pitch; language is retained in the model but not sent as a separate request field.

Main functions:

- `voices(...)`: Returns official built-in voices for this model, independent of network or credentials.
- `resolve(...)`: Resolves default XML voices and rejects obsolete local IDs or unsupported options clearly.
- `preflight(...)`: Validates local settings without an HTTP request or charge.
- `synthesize(...)`: Synthesizes one cue; returned bytes have passed WAV validation and exact-length normalization.
- `testConnection(...)`: Checks authentication and selected-model access without generating billable audio.
- `getBody(...)`: Publishes only the complete response body.
- `onSubscribe(...)`: Requests chunks as they arrive from the Java HTTP client.
- `onNext(...)`: Enforces a 64 MiB per-cue response bound before allocating another chunk.
- `onError(...)`: Completes a failed transfer without exposing request details.
- `onComplete(...)`: Returns the complete response once transport has finished.

9.4.6 OpenAIVoices

Source: `audio/OpenAIVoices.java`

Provides the compiled speech model and voice catalogue used by this source snapshot, including default model and voice constants. It does not query a live service catalogue.

Authoring note: Compiled catalogue in this snapshot, not a live service lookup. Do not treat source comments about verification dates as a guarantee of current API availability.

Main functions:

- `voices(...)`: Returns the official voice subset for the selected model; no API key or discovery needed.

9.4.7 SpeechCache

Source: `audio/SpeechCache.java`

Content-addressed PCM WAV cache. Writes are staged and atomically published when supported. Cache hits are decoded and duration-checked too; corrupt entries are removed and regenerated.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `key(...)`: Hashes length-prefixed request fields to avoid delimiter ambiguity.
- `resolve(...)`: Obtains valid speech, then rejects any cue whose actual duration exceeds its narration window.
- `clear(...)`: Deletes only content-addressed WAV entries; unrelated files are preserved.

9.4.8 SpeechProvider

Source: `audio/SpeechProvider.java`

Injectable narration boundary. Unit tests can supply WAV fixtures without network access.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `resolve(...)`: Resolves a default voice before hashing and synthesis; fixtures may use settings unchanged.
- `synthesize(...)`: Returns complete PCM WAV bytes, including a WAV header; never an MP3 payload.

9.4.9 SpeechVoice

Source: `audio/SpeechVoice.java`

A documented OpenAI voice identity used consistently by the GUI and CLI.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions: The type supplies the data, contract or internal operations described above; it is not a separate screenplay command.

9.4.10 VoiceSettings

Source: `audio/VoiceSettings.java`

Immutable speech synthesis settings. Credentials are deliberately absent from this model. The OpenAI provider validates supported controls.

XML connection: voice-related speak attributes.

Main functions:

- `defaults(...)`: Returns the British English voice used by the production lessons.

9.4.11 WavAudio

Source: `audio/WavAudio.java`

Validates RIFF/WAVE and rewrites complete audio as signed 16-bit PCM with exact chunk sizes. Duration uses actual data bytes, never an untrusted Java Sound frame-count/header estimate. Streaming data lengths `0xffffffff` and `0x7fffffff` are resolved using received payload length. Sample rate and channel count are retained; AudioMixer performs the single 48 kHz conversion.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `normalize(...)`: Validates container, chunk lengths and PCM/float format, then creates an ordinary PCM WAV.

9.5 Package cli (4 files)

9.5.1 GaussianEliminationMain

Source: cli/GaussianEliminationMain.java

Builds an authored Gaussian-elimination lesson and exposes its complete XML through screenplay. main supports preview, no-speech, write-only and single-frame workflows; full narrated output requires configured speech and FFmpeg.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `main(...)`: Creates and validates the lesson, then renders the requested output.
- `screenplay(...)`: Builds the complete timed XML, calculating every matrix state from the row operations.

9.5.2 Main

Source: cli/Main.java

Production CLI with deterministic frame/range sampling and explicit editorial audio mode. Errors exit with status 2. Existing outputs are never replaced implicitly.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `main(...)`: Runs the CLI, converting failures into concise diagnostics and status 2.
- `run(...)`: Parses a command; programmatic callers receive the original exception.
- `stage(...)`: Prints real stage transitions for automated jobs.
- `part(...)`: Samples real chapter/global time for terminal progress.
- `frame(...)`: Receives a completed engine frame and its measured production counters.
- `speech(...)`: Receives actual speech-cache provenance and measured clip duration.
- `completed(...)`: Shows measured output size and exposes local playback, folder opening and path copying.

9.5.3 RenderLesson

Source: cli/RenderLesson.java

Minimal XML-to-PNG-to-MP4 entry point with independent input, frames and output paths. Loads one lesson, exports its numbered PNG sequence, then invokes FFmpeg. With no arguments it uses the bundled matrix showcase; three arguments select XML, frame directory and MP4 paths.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `main(...)`: Renders XML to PNG frames and then encodes MP4 using optional explicit paths.

9.5.4 SudokuBacktrackingMain

Source: cli/SudokuBacktrackingMain.java

Builds a 78-second Sudoku demonstration from a real recursive solver trace, including failed placements and reversals. screenplay produces editable XML; main can write XML, render a still or build a video with configured speech/FFmpeg.

Authoring note: Java generates a genuine backtracking trace and then writes supported XML. The solver itself needs no AI; narrated production still uses the configured speech provider.

Main functions:

- `main(...)`: Generates XML and optionally a PNG or narrated MP4 in a new output/sudoku-* folder.
- `screenplay(...)`: Creates the complete editable screenplay from a verified recursive search trace.

9.6 Package editor (1 files)

9.6.1 XmlEditor

Source: `editor/XmlEditor.java`

EduVideo XML editor with undo history, line numbers, lexical highlighting and literal find/replace.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `load(...)`: Loads a document and starts a fresh undo history.
- `focusFind(...)`: Focuses the search field without stealing the editor's selection.
- `findNext(...)`: Selects the next literal match, wrapping once.
- `jump(...)`: Navigates to a one-based parser error line, clamped to current text.
- `problem(...)`: Marks a one-based parser error line without interfering with XML token highlighting.

9.7 Package export (3 files)

9.7.1 FFmpegExporter

Source: `export/FFmpegExporter.java`

Optional silent H.264 export. FFmpeg runs directly, without a shell. Creates a `ProcessBuilder` argument list directly, preserving paths with spaces. It pads odd dimensions, requests H.264/yuv420p and fast-start metadata, refuses output overwrites and retains PNGs on failure.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `command(...)`: Builds a shell-free FFmpeg argument list for silent H.264/yuv420p output.
- `export(...)`: Runs FFmpeg and checks its exit status; PNG input is preserved on failure.

9.7.2 FrameExporter

Source: `export/FrameExporter.java`

Writes one image at a time. Refuses stale frame sequences in the output directory. Streams one frame at a time to six-digit numbered PNG files. It refuses directories with prior frame files so FFmpeg cannot consume a stale mixed sequence.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `export(...)`: Streams a numbered PNG sequence, refusing stale frame files.

9.7.3 ProductionBuilder

Source: `export/ProductionBuilder.java`

Preflight-first production pipeline. Speech is generated, cached, measured and mixed before visual rendering. BGR frames stream directly to FFmpeg, avoiding a huge PNG sequence. Temporary products stay beside the destination and are removed after success, failure or cancellation.

Authoring note: Use for narrated production. The older `FFmpegExporter` path is silent. Profiles choose actual raster size/fps.

Main functions:

- `build(...)`: Builds a protected MP4 and a report.

9.8 Package math (16 files)**9.8.1 AreaPlot**

Source: `math/AreaPlot.java`

Shaded finite area to $y=0$ or midpoint Riemann rectangles; discontinuous sampled strips are omitted. Samples a fixed expression between ordered finite bounds. Zero rectangles requests a shaded area; positive counts use midpoint rectangles. `drawProgress` reveals the interval left to right. This is a visualization, not numerical integration.

XML connection: `area`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.8.2 CalculusObject

Source: `math/CalculusObject.java`

Live calculus construction inside a coordinate plane. `PARAM_A` controls secant displacement or Riemann rectangle count. Function evaluation and labels use the same mathematical samples. The derivative is a central numerical difference, explicitly distinct from symbolic algebra.

XML connection: `calculus`.

Main functions:

- `slope(...)`: Returns the current secant slope, switching to a central difference at zero displacement.

9.8.3 CoordinatePlane

Source: `math/CoordinatePlane.java`

Cartesian coordinates in a bounded pixel rectangle; child plots share this transform. Maps mathematical x/y coordinates to a bounded pixel rectangle, reversing y for screen space. Functions, areas and plot points must be direct children; other children retain ordinary pixel coordinates.

XML connection: `axes`.

Main functions:

- `bounds(...)`: Returns the container viewport rather than the union of overflowing child content.
- `anchor(...)`: Resolves `math:x:y` coordinates through the animated plane dimensions.
- `appearance(...)`: Configures grid opacity, tick typography and an optional axis colour override.
- `labels(...)`: Sets the horizontal and vertical axis labels.
- `screenX(...)`: Maps a mathematical x -coordinate to local pixel space.
- `screenY(...)`: Maps a mathematical y -coordinate to local pixels, reversing the vertical direction.
- `mathX(...)`: Inverts the horizontal mathematical-to-pixel mapping.

9.8.4 EquationObject

Source: `math/EquationObject.java`

Unicode equation with an injectable typesetting backend. Retains an injectable `EquationRenderer` strategy. Plain Unicode is backward compatible; opt-in structured typesetting supports fractions, roots and scripts. Structured reveal fades the whole equation to avoid cutting markup mid-command.

XML connection: `equation`.

Main functions:

- `bounds(...)`: Uses typeset metrics for fractions and scripts when the mathematical backend is selected.
- `typeset(...)`: Selects compact mathematical markup rendering or backward-compatible Unicode text.
- `renderer(...)`: Replaces the equation layout strategy while retaining the object timeline.

9.8.5 FunctionPlot

Source: `math/FunctionPlot.java`

Pixel-density sampling with midpoint checks and gaps at poles/non-finite regions. Compiles a safe arithmetic expression once and samples approximately twice per output pixel. Non-finite values, steep jumps and midpoint anomalies break the path rather than joining across poles. Parameters *a*, *b* and *c* can be animated.

XML connection: `function`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.8.6 GeometryObject

Source: `math/GeometryObject.java`

Arbitrary immutable copied Java2D geometry; polygons, arcs, angles and paths. Copies the supplied shape as a `Path2D`. Filled and outlined geometry share `ShapeObject` animation, glow and dashed-stroke behaviour, so camera transforms remain consistent.

XML connection: `polygon`, `path`, `ray`, `arc`, `angle`.

Main functions:

- `polygon(...)`: Configures polygon for this component.

9.8.7 MatrixAlgebra

Source: `math/MatrixAlgebra.java`

Small dense numerical matrix operations for authoring validated explanations. Inputs are copied; methods never mutate a caller's matrix. Singular pivots use a $1e-12$ threshold and results are double precision, not symbolic or exact rational arithmetic.

Authoring note: Java authoring utility only: there is no matrix-algebra, determinant, `rref` or row-operation XML instruction. Compute labels before writing the screenplay.

Main functions:

- `identity(...)`: Creates the square identity matrix of positive order.
- `add(...)`: Adds matrices with equal dimensions.
- `scale(...)`: Multiplies every cell by a finite scalar.
- `transpose(...)`: Transposes rows and columns into a new matrix.
- `multiply(...)`: Multiplies conformable matrices; vectors are represented by single-column matrices.
- `swapRows(...)`: Swaps two rows without changing the source matrix.
- `addRow(...)`: Performs `target += factor * source`, the elementary Gaussian row-addition operation.
- `determinant(...)`: Computes a square determinant by pivoted Gaussian elimination.
- `rref(...)`: Computes reduced row echelon form using partial pivoting.

9.8.8 MatrixObject

Source: `math/MatrixObject.java`

Rectangular immutable labels with independently timed highlights and cell visibility. All times are scene-local seconds. Copies rectangular cell labels and stores visibility changes in per-cell ordered maps. Row/column/cell highlights use half-open time intervals; the last configured overlapping highlight wins. All indices are zero-based and independent of render order.

XML connection: `matrix`; `matrix highlight/visibility` events.

Main functions:

- `bounds(...)`: Returns the matrix's complete cell-grid bounds in local pixels.
- `anchor(...)`: Accepts `cell:r:c`, `row:r` and `column:c` anchors with zero-based indices.
- `cell(...)`: Returns the immutable label at a zero-based row and column.
- `cells(...)`: Configures cell dimensions and label font size in local pixels.
- `initiallyVisible(...)`: Sets default cell visibility before any per-cell visibility event.
- `highlightRow(...)`: Adds a highlight across a zero-based matrix row during `[start,start+duration)`.
- `highlightColumn(...)`: Adds a highlight down a zero-based matrix column during `[start,start+duration)`.
- `highlightCell(...)`: Adds a highlight to one matrix cell during `[start,start+duration)`.
- `revealCell(...)`: Makes one matrix cell visible from the specified time until a later visibility event.
- `hideCell(...)`: Hides one matrix cell from the specified time until a later visibility event.
- `isCellVisible(...)`: Samples the last cell visibility event at or before time without changing state.
- `highlightAt(...)`: Samples the last configured active row, column or cell highlight.

9.8.9 MatrixProductStep

Source: `math/MatrixProductStep.java`

Reusable explanatory row-by-column operation. Copies operands from matrix anchors, simplifies products, gathers the sum and flies it into the destination cell. Sources remain visible; transient working content retires automatically. Numeric labels are checked against the authored result so a beautiful animation cannot hide wrong arithmetic.

XML connection: `matrix-product-step` event.

Main functions:

- `result(...)`: Returns the independently computed result, useful for verification and captions.

9.8.10 NumberLine

Source: `math/NumberLine.java`

Horizontal labeled real-number axis. Draws a horizontal axis with evenly spaced numerical ticks. The mathematical range is fixed at construction while pixel width can be animated; excessive tick counts are rejected.

XML connection: `number-line`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.8.11 PlotPoint

Source: `math/PlotPoint.java`

Labelled mathematical point in a coordinate plane. Use for roots, intersections and extrema supplied by a lesson. Stores coordinates in mathematical units and resolves them through its axes parent at each sample. Labels use pixel offsets, and optional dashed guides project onto $x=0$ and $y=0$. Root finding is performed by the lesson author.

XML connection: `plot-point`.

Main functions:

- `labelOffset(...)`: Positions the point label relative to its marker in pixels.

9.8.12 StatisticsPlot

Source: `math/StatisticsPlot.java`

Deterministic data plot with live sample count, bar/histogram/scatter views and a fitted normal-density curve. Labels report population variance and standard deviation. The displayed 95% mean interval uses the normal approximation, not an exact small-sample CI.

XML connection: `statistics`.

Main functions:

- `statistics(...)`: Returns mean, median, population variance and population standard deviation.

9.8.13 TokenEquation

Source: `math/TokenEquation.java`

Structured equation with stable token identities. Matching tokens move continuously; removed tokens shrink away and new tokens grow in. This is a presentation model, not a CAS.

XML connection: `token-equation`; `transform-equation` event.

Main functions:

- `parse(...)`: Parses semicolon-separated `identity=label` pairs, preserving equals signs in labels.
- `transform(...)`: Adds a nonoverlapping transformation; edits must be configured in chronological order.
- `bounds(...)`: Returns conservative baseline-relative bounds of the current token layout.
- `anchor(...)`: Resolves `term:identity` to its live interpolated baseline location.

9.8.14 TriangleObject

Source: `math/TriangleObject.java`

Labelled triangle with geometrically computed angle arcs; independent side/angle emphasis uses bit masks. Copies three non-collinear vertices and computes interior angles from their vectors. Side labels follow consecutive edges, while masks select static emphasis. `highlight` and `angleHighlight` use 1-based selections (zero means none).

XML connection: `triangle`.

Main functions:

- `anchor(...)`: Returns `vertex:index` or `vertex:label` coordinates for geometry transfers and construction marks.
- `bounds(...)`: Computes the actual triangle's axis-aligned local bounds.
- `emphasis(...)`: Selects independently emphasized edges and angles using three-bit masks.
- `angle(...)`: Computes an interior angle in degrees from the two incident vertex vectors.

9.8.15 VectorDiagram

Source: `math/VectorDiagram.java`

Animated vector addition, projection, complex multiplication and linear grid transformation. Coordinates use mathematical units in a local centred viewport. `PARAM_A` in $[0,1]$ controls construction progress. Linear maps interpolate from identity; complex maps interpolate polar rotation and modulus so multiplication visibly rotates and scales.

XML connection: `vectors`.

Main functions:

- `mapped(...)`: Interpolates a target linear map from identity.

9.8.16 VennDiagram

Source: `math/VennDiagram.java`

Two-set union, intersection or difference drawn with exact Java2D area operations. `DRAW_PROGRESS` reveals the selected region while set outlines preserve spatial correspondence.

XML connection: `venn`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9 Package objects (11 files)

9.9.1 ArrowObject

Source: `objects/ArrowObject.java`

Arrowhead tracks the progressively drawn endpoint. Adds a filled arrowhead to `LineObject`. The head follows the visible endpoint as `drawProgress` changes and is omitted when progress is zero.

XML connection: `arrow`, `vector`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9.2 CircleObject

Source: `objects/CircleObject.java`

Circle or ellipse centered on its local origin. Treats width and height as diameters around the local origin. Equal values produce a circle; independent dimensions produce an ellipse.

XML connection: `circle`, `ellipse`, `point`.

Main functions:

- `bounds(...)`: Returns centred local shape bounds, consistent with the drawing origin.

9.9.3 CurvedArrowObject

Source: `objects/CurvedArrowObject.java`

Quadratic callout arrow with a tangent-aligned head and deterministic draw progress. Uses one quadratic control point and a signed endpoint. The arrowhead follows the tangent of the flattened visible path, so progressive drawing remains aligned with the curve.

XML connection: `curved-arrow`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9.4 ImageObject

Source: `objects/ImageObject.java`

Image decoded once at construction; caller must not mutate the source raster. Decodes supported ImageIO formats once from a local path and initializes dimensions to native size. Rendering can scale the raster through animated width/height and the normal object transform.

XML connection: `image`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9.5 LineObject

Source: `objects/LineObject.java`

Segment from origin to a signed local width,height endpoint. Allows negative width and height because they represent the local endpoint rather than bounds. ShapeObject supplies partial drawing, dashed outlines and glow.

XML connection: `line`, `segment`.

Main functions:

- `signedDimensions(...)`: Reports whether width and height represent signed endpoints instead of nonnegative bounds.

9.9.6 PanelObject

Source: `objects/PanelObject.java`

Rounded container whose child coordinates start inside its padding. Uses the normal group timeline. Owns nested scene content while drawing its surface in outer coordinates. Padding translates child origins but does not clip overflow; explicit width/height and child layout remain under author control.

XML connection: `panel`.

Main functions:

- `bounds(...)`: Returns the container viewport rather than the union of overflowing child content.
- `padding(...)`: Returns the content translation applied before children are drawn.
- `surface(...)`: Configures gradient colours, rounded corners, border and shadow.

9.9.7 RectangleObject

Source: `objects/RectangleObject.java`

Rectangle with top-left local origin. Uses a top-left origin and nonnegative animated dimensions. Fill, progressive outlines, dashed strokes and glow are inherited from ShapeObject.

XML connection: `rectangle`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9.8 ResultObject

Source: `objects/ResultObject.java`

Semantic answer card with centered text, inherited opacity and animatable glow/scale. Chooses accent colours from SUCCESS, ANSWER, WARNING or INFO semantics and centers the answer within a rounded card. Normal opacity, scale and glow properties can emphasize the result.

XML connection: `result`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9.9 ShapeObject

Source: `objects/ShapeObject.java`

Shared fill and progressively drawn outline behavior. Provides a template method for time-sampled geometry. Fill occurs after complete draw progress; before completion the outline is revealed. Dashed strokes are rendered without multi-pass glow to retain dash spacing.

Authoring note: Abstract rendering base, not an XML tag. Its dashed/fill behaviour is exposed by appropriate concrete shape tags.

Main functions:

- `dashed(...)`: Enables dashed construction strokes on shape outlines.
- `fill(...)`: Selects filled geometry or an outlined shape.

9.9.10 StepObject

Source: `objects/StepObject.java`

Instruction card with a numbered badge and heading; place child content below `y=40` inside padding. Adds a numbered badge and instructional heading to a `PanelObject`. Children use padded coordinates; reserve the first 40 pixels for the header. Animate the group for coordinated entrance effects.

XML connection: `step`.

Main functions: Construction/configuration and the inherited or record-generated accessors support the role above. Rendering subclasses draw through the inherited scene pipeline.

9.9.11 TextObject

Source: `objects/TextObject.java`

Baseline-aligned Unicode text with character or word reveal. Stores immutable text with font, alignment and character/word reveal configuration. Character reveal respects Unicode code points; multi-line text advances by 1.3 times the font size.

XML connection: `text`; typewriter event.

Main functions:

- `bounds(...)`: Measures multiline baseline-aligned bounds using the configured physical font metrics.
- `weight(...)`: Selects bold or regular text weight.
- `style(...)`: Configures text family, positive font size and left/center/right alignment.
- `words(...)`: Selects word-based instead of Unicode code-point reveal.
- `revealed(...)`: Returns the text prefix visible at the requested scene-local time.

9.10 Package preview (1 files)

9.10.1 PreviewApp

Source: `preview/PreviewApp.java`

Swing shell around the same renderer; expensive work happens outside the EDT. Owns Swing controls on the event dispatch thread and delegates parsing/rendering to workers. Generation tokens discard stale results after a reload, and pending requests coalesce during scrubbing.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `windowClosed(...)`: Stops the playback timer when the preview window closes.
- `doInBackground(...)`: Performs parsing or rendering away from the Swing event dispatch thread.
- `done(...)`: Publishes the completed worker result only when its generation is still current.
- `open(...)`: Opens the desktop preview on the Swing event dispatch thread.

9.11 Package production (8 files)

9.11.1 Cancellation

Source: `production/Cancellation.java`

Per-job cancellation token and subprocess owner. No render shares another render's processes.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `isCancelled(...)`: Reports whether this job was explicitly cancelled.
- `finish(...)`: Releases all owned subprocesses after a failed or finished job.
- `check(...)`: Rejects work after cancellation or interruption.
- `cancel(...)`: Stops owned processes, including descendants, so a blocked encoder pipe is released.
- `own(...)`: Registers a newly started child and closes the start/cancel race.
- `release(...)`: Releases an exited process reference.
- `terminateAndWait(...)`: Terminates and reaps an owned process on the worker before its files may be removed.
- `run(...)`: Runs a command without a shell, capturing bounded diagnostics and enforcing a timeout.

9.11.2 FrameSequence

Source: `production/FrameSequence.java`

Shared single/multipart render input. A worker retains only the current chapter's renderer/backdrops.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `atOutput(...)`: Creates a separate worker-owned renderer/cache for another output raster.
- `audioTimeline(...)`: Returns the shared globally timed audio timeline.
- `validate(...)`: Validates all immutable lesson snapshots.
- `lessons(...)`: Returns lesson models for reporting without flattening their scene namespaces.
- `partIndex(...)`: Returns a chapter index for a global time.
- `progress(...)`: Emits chapter progress using actual sampled frame ranges, including partial renders.
- `renderFrame(...)`: Renders one actual global sample through the unchanged graphics engine at chapter-local time.

9.11.3 PartProgress

Source: `production/PartProgress.java`

Actual chapter frame counters and both clocks, emitted alongside the global frame counter.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `fraction(...)`: Fraction of this chapter's requested frames submitted to the encoder.

9.11.4 ProductionAudio

Source: `production/ProductionAudio.java`

Composes clocks once, before mixing, so a music region has only one decoder and one fade envelope.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `assemble(...)`: Offsets narration and effects; joins touching, equivalent music beds across chapter boundaries.

9.11.5 ProductionListener

Source: `production/ProductionListener.java`

Engine events delivered on the production worker. Listeners must return quickly and never mutate frames.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `stage(...)`: Reports an actual pipeline transition, without inventing overall stage percentages.
- `frame(...)`: Reports a completed immutable frame; consumers should retain at most a bounded sample.
- `part(...)`: Reports actual chapter counters alongside global frame progress.
- `speech(...)`: Reports cache outcome and measured duration for a narration cue.
- `message(...)`: Reports a diagnostic with a category suitable for filtering.
- `completed(...)`: Reports a successfully published output.

9.11.6 ProductionProgress

Source: `production/ProductionProgress.java`

Measured frame-stage statistics. ETA is withheld until at least one frame has completed.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `fraction(...)`: Fraction of video frames actually submitted to the encoder, not an overall pipeline estimate.
- `fps(...)`: Mean completed-frame throughput since visual rendering began.
- `remaining(...)`: Estimated remaining frame-stage seconds; NaN denotes insufficient observations.

9.11.7 ProductionProject

Source: `production/ProductionProject.java`

Validated ordered lesson snapshots. Source XML clocks and scene identities are never rewritten.

XML connection: `edu-production` root.

Main functions:

- `global(...)`: Converts a local timestamp to production time without modifying the lesson.
- `partIndex(...)`: Finds the chapter containing a production timestamp; a boundary belongs to the next chapter.
- `audioTimeline(...)`: Builds globally timed narration/effects and continuous compatible music regions.
- `chapters(...)`: Creates an optional human-readable chapter list with millisecond precision.

9.11.8 RenderQueue

Source: `production/RenderQueue.java`

Sequential production queue; each item owns an immutable validated snapshot and distinct cancellation token.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `cancel(...)`: Cancels the active item; waiting items remain queued and sequential processing continues.

9.12 Package project (2 files)

9.12.1 ProjectLibrary

Source: `project/ProjectLibrary.java`

Filesystem discovery of ordinary XML lessons. Never follows directory symlinks or modifies projects.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `discover(...)`: Returns sorted XML files below the selected root, omitting generated build/cache trees.
- `preVisitDirectory(...)`: Skips generated output and cache directories during project discovery.
- `visitFile(...)`: Collects XML candidates without attempting a potentially expensive parse during traversal.
- `save(...)`: Saves an editor snapshot atomically to avoid truncation on failure.

9.12.2 ProjectTemplates

Source: `project/ProjectTemplates.java`

Resource-backed project templates in the parser's real edu-video vocabulary.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `create(...)`: Expands safe scalar placeholders without inventing another project format.

9.13 Package render (9 files)

9.13.1 BackgroundRenderer

Source: `render/BackgroundRenderer.java`

Configurable, deterministic procedural backdrop; decoded images are optional and never required. Combines a linear base, radial illumination, optional image, perspective grid, curves, fixed dots and vignette. Patterns are deterministic and screen-fixed. Images decode during configuration rather than once per frame.

XML connection: scene background child.

Main functions:

- `options(...)`: Configures deterministic backdrop layers and clamped effect intensity.
- `image(...)`: Decodes an optional background raster once during configuration.

9.13.2 EquationRenderer

Source: `render/EquationRenderer.java`

Rendering strategy interface for equation text. Implementations draw an equation using the supplied graphics and alignment; EquationObject can receive a custom implementation in Java.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `draw(...)`: Draws equation text at the local baseline using the caller font, colour and alignment.

9.13.3 GlowRenderer

Source: `render/GlowRenderer.java`

Reusable multi-pass outline lighting in local coordinates, preserving inherited opacity. Draws broad low-alpha strokes followed by a crisp core. It preserves caller graphics state and inherited composite alpha; all distances are local coordinates and therefore follow camera scale.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `alpha(...)`: Returns a colour with its alpha multiplied by a clamped opacity factor.

9.13.4 JavaHighlighter

Source: `render/JavaHighlighter.java`

Basic Java keywords, strings, numeric literals and line comments. Recognizes Java keywords, numeric literals, quoted strings/chars and line comments. It intentionally does not implement a complete Java lexer or multi-line lexical state.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `highlight(...)`: Recognizes Java tokens while preserving intervening text and whitespace.

9.13.5 MathRenderer

Source: `render/MathRenderer.java`

Small dependency-free equation layout for Unicode, fractions, roots, and braced scripts. Not a full LaTeX interpreter. Builds lightweight layout boxes for Unicode plus $\frac{a}{b}$, $\sqrt{x}$, $^{\text{superscripts}}$ and $_{\text{subscripts}}$. Box metrics reserve ascent/descent for fractions. This intentionally remains a small subset, not a general LaTeX engine.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `measure(...)`: Measures a typeset expression for semantic bounds and equation alignment.
- `width(...)`: Returns the horizontal advance of this mathematical layout box.
- `above(...)`: Returns the box extent above its baseline.
- `below(...)`: Returns the box extent below its baseline.
- `row(...)`: Parses a row of mathematical boxes, optionally stopping at a closing brace.
- `validate(...)`: Rejects unbalanced braces in structured mathematical text.

9.13.6 PanelRenderer

Source: `render/PanelRenderer.java`

Shared card surface renderer. Paints shadow, gradient, border and optional restrained glow. Centralizes rounded gradient fills, restrained offset shadows and outline glow. It uses child graphics state and does not reposition scene children; `PanelObject` supplies content padding.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions: The type supplies the data, contract or internal operations described above; it is not a separate screenplay command.

9.13.7 PathRenderer

Source: `render/PathRenderer.java`

Arc-length reveal, preserving subpath boundaries and discontinuities. Flattens curves with a local-space tolerance and accumulates segment lengths. Partial paths keep separate moves at segment boundaries, preventing accidental links between discontinuous subpaths.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `partial(...)`: Returns the initial fraction of a flattened path measured by arc length.

9.13.8 SyntaxHighlighter

Source: `render/SyntaxHighlighter.java`

Interface for source-preserving coloured text spans. `highlight` returns tokens whose concatenated text reproduces the source line; its nested `Token` record carries text and colour.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `highlight(...)`: Tokenizes one source line into ordered, source-preserving coloured spans.

9.13.9 TextRenderer

Source: `render/TextRenderer.java`

Unicode text layout with baseline positioning. Uses `TextLayout` for antialiased Unicode glyph placement and fractional advances. Positions describe the text baseline; alignment adjusts horizontal origin without changing content.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions: The type supplies the data, contract or internal operations described above; it is not a separate screenplay command.

9.14 Package scene (7 files)

9.14.1 Anchor

Source: `scene/Anchor.java`

A validated semantic connection to a scene object, evaluated at the requested scene time.

Authoring note: Resolved from `object@anchor` references in supported events. There is no anchor object element.

Main functions:

- `at(...)`: Samples the connection in scene coordinates.
- `resolve(...)`: Resolves `object@anchor` notation; a bare identity selects its centre.

9.14.2 Camera

Source: `scene/Camera.java`

Camera `x/y` are pan offsets; zoom and rotation pivot at canvas center. The renderer applies this transform to scene objects after drawing the screen-space background. Pan is measured in scene pixels, rotation in degrees, and zoom must stay positive.

XML connection: built-in target camera; camera events (no camera object tag).

Main functions:

- `transform(...)`: Configures transform for this component.

9.14.3 GroupObject

Source: `scene/GroupObject.java`

Nested local coordinate system with inherited opacity and stable z ordering. Enforces single ownership and rejects cycles. Children render in stable z-order inside the group transform; inherited opacity composes multiplicatively.

XML connection: `group`.

Main functions:

- `bounds(...)`: Unions transformed child bounds in this group's local coordinate system.
- `add(...)`: Attaches an unparented child, rejecting self-parenting and group cycles.

9.14.4 Layout

Source: `scene/Layout.java`

Optional configure-time layout helpers; explicit object positions remain the default. Computes positions once during scene construction using configured width and height. It does not continuously reflow animated objects; text baseline positioning remains explicit.

Authoring note: Java-only configure-time layout. There is no automatic horizontal/vertical layout XML tag.

Main functions:

- `horizontal(...)`: Places objects consecutively using initial widths and a nonnegative gap.
- `vertical(...)`: Places objects consecutively using initial heights and a nonnegative gap.
- `center(...)`: Centers an object using its configured initial dimensions.

9.14.5 SafeLayout

Source: `scene/SafeLayout.java`

Resolution-independent authoring layout with a 5% safe inset and explicit content regions.

Authoring note: Java-only layout regions/grid helper. XML must contain the resulting explicit coordinates.

Main functions:

- `title(...)`: Returns the title-safe rectangle.
- `work(...)`: Returns the main mathematical working region.
- `result(...)`: Returns the lower conclusion region above the safe bottom margin.
- `grid(...)`: Places objects into evenly distributed cells; gaps are in logical pixels.

9.14.6 Scene

Source: `scene/Scene.java`

A scene with its own clock, object namespace, and incoming transition. Owns a root group, camera, optional theme and backdrop, and transition configuration. Validation checks unique IDs, event bounds, plot parents, graph endpoints and tree topology before rendering.

XML connection: `scene`.

Main functions:

- `backdrop(...)`: Configures backdrop for this component.
- `add(...)`: Attaches a top-level visual object to this scene root.
- `objects(...)`: Builds an immutable scene-wide lookup including nested objects and the camera.

- `find(...)`: Resolves an object ID or rejects an unknown target.
- `validate(...)`: Checks IDs, event duration bounds, plot placement, edge endpoints and tree topology.

9.14.7 SceneObject

Source: `scene/SceneObject.java`

Base visual object. Configure before rendering; sampling never mutates properties. Owns identity, base properties, timeline, parent and z-order. Rendering creates an isolated Graphics2D context, combines inherited opacity and applies local translation, rotation and scale before drawing. Configure instances before starting a render.

Authoring note: Abstract visual base, not an XML tag. All normal visual objects inherit its ID, properties, transforms and timeline.

Main functions:

- `dim(...)`: Dims this object during a half-open interval with smooth 200 ms edges.
- `bounds(...)`: Returns local rectangular bounds; specialised shapes override their natural origin.
- `anchor(...)`: Resolves a named local anchor.
- `worldTransform(...)`: Returns this object's complete local-to-scene transform, including panel content insets.
- `worldAnchor(...)`: Resolves an anchor in scene coordinates, independent of camera and output resolution.
- `visible(...)`: Enables or disables rendering of this object and its descendants.
- `signedDimensions(...)`: Whether width/height represent signed endpoints instead of object bounds.
- `set(...)`: Sets an initial property.
- `at(...)`: Sets the initial local position in pixels.
- `color(...)`: Sets the initial colour or samples its animated ARGB value.
- `value(...)`: Samples a property at scene-local time without modifying this object.
- `animate(...)`: Appends a non-overlapping two-keyframe track; first/last values hold outside its interval.
- `render(...)`: Renders in a child graphics context, composing inherited alpha and transforms.

9.15 Package ui (10 files)

9.15.1 LogPane

Source: `ui/LogPane.java`

Bounded categorized production log. All methods run on the JavaFX thread.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `add(...)`: Appends a real event and discards the oldest entries at the retention limit.

9.15.2 MainWindow

Source: `ui/MainWindow.java`

Workspace coordinator. Components own their UI; parsing, preview, speech and production share engine services.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `updateItem(...)`: Displays the filesystem name while allowing JavaFX to recycle explorer cells.
- `shortcuts(...)`: Installs editing-aware keyboard shortcuts after a Scene is available.

- `open(...)`: Opens disk XML after offering to preserve unsaved changes.

9.15.3 PreferencesDialog

Source: `ui/PreferencesDialog.java`

Studio preferences with one OpenAI narration form and the existing local video tools.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `show(...)`: Opens a draft settings dialog; previews use draft values and Save commits them.

9.15.4 PreviewPane

Source: `ui/PreviewPane.java`

Actual engine preview with a dedicated serial renderer, bounded LRU frames and coalesced seeks.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `handle(...)`: Samples pending state on a JavaFX animation pulse without queuing every engine frame.
- `production(...)`: Previews the complete manifest using its global clock and real chapter renderers.
- `seek(...)`: Requests a clamped source time; only the newest pending request is rendered.
- `toggle(...)`: Toggles visual playback; final mixed audio is auditioned through completed video output.
- `step(...)`: Moves exactly one source frame per direction unit.
- `fullscreen(...)`: Shows the currently displayed actual image in a separate fullscreen stage.

9.15.5 ProductionPane

Source: `ui/ProductionPane.java`

Latest-frame mailbox sampled at 10 Hz. Full-resolution frames are never queued onto JavaFX.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `onProgress(...)`: Receives measured status text on JavaFX at the monitor refresh rate.
- `handle(...)`: Samples pending state on a JavaFX animation pulse without queuing every engine frame.
- `onCancel(...)`: Sets the cancellation action for the queue's current job.
- `reset(...)`: Clears previous job statistics before new production begins.
- `stage(...)`: Records the latest actual stage.
- `frame(...)`: Replaces the frame mailbox, retaining only the most recent engine image.
- `part(...)`: Retains only the most recent actual chapter counters for the next UI pulse.
- `speech(...)`: Records the actual cache result and measured speech duration.
- `message(...)`: Adds a bounded diagnostic without blocking production.
- `completed(...)`: Records completion after the final output has been published.

9.15.6 ProductionPartsPane

Source: `ui/ProductionPartsPane.java`

Chapter management edits the authoritative XML through the editor's undo history. No parallel document is saved.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `updateItem(...)`: Displays both chapter-local duration and its production start.
- `time(...)`: Formats a source duration for chapter lists without losing hours.

9.15.7 ProjectDialogs

Source: `ui/ProjectDialogs.java`

Focused project and delivery dialogs that expose only supported engine parameters.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `create(...)`: Collects a template and logical authoring dimensions, validating scalars before closing.
- `render(...)`: Configures a real output profile and source range; existing files remain protected by the engine.
- `renderProduction(...)`: Configures a multipart job through the same profile, voice, range and queue controls.

9.15.8 TimelinePane

Source: `ui/TimelinePane.java`

Read-only scene, speech and audio tracks derived directly from the validated engine model.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `production(...)`: Shows chapter boundaries and already-offset speech/music on the complete production clock.

9.15.9 Ui

Source: `ui/Ui.java`

Shared compact controls and expandable diagnostic dialogs for a consistent desktop interface.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `button(...)`: Creates a toolbar/action button with a tooltip.
- `panel(...)`: Creates a named panel with external CSS styling.
- `error(...)`: Explains the failure first; stack traces remain behind the Details expander.
- `confirm(...)`: Asks for confirmation before deleting or discarding user work.

9.15.10 VoicePane

Source: `ui/VoicePane.java`

Shared OpenAI narration form for the sidebar and Preferences, with cancellable background previews.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `refresh(...)`: Reloads saved/session settings when Preferences changes the active configuration.

- `save(...)`: Commits the form; credential persistence occurs only when Remember was explicitly selected.

9.16 Package util (4 files)

9.16.1 Checks

Source: `util/Checks.java`

Shared finite-number and range validation. Provides finite, positive and nonnegative checks with contextual error labels. Validation methods return the checked value for assignment; clamp only bounds a value to [0,1].

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `finite(...)`: Rejects non-finite numbers and returns the checked value.
- `positive(...)`: Rejects non-finite or nonpositive numbers and returns the checked value.
- `nonnegative(...)`: Rejects non-finite or negative numbers and returns the checked value.
- `clamp(...)`: Bounds a numeric value to the closed interval [0,1].

9.16.2 ColorParser

Source: `util/ColorParser.java`

Parses unambiguous RGB and RGBA hexadecimal literals. Accepts only six-digit RGB or eight-digit RGBA hexadecimal literals. Interpolation operates on individual channels and stores ARGB exactly as a double for timeline compatibility.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `parse(...)`: Parses strict `#RRGGBB` or `#RRGGBBAA` colour notation.
- `mix(...)`: Interpolates red, green, blue and alpha channels independently.

9.16.3 ExpressionParser

Source: `util/ExpressionParser.java`

Recursive descent arithmetic compiler. No scripting engine or arbitrary code execution. Compiles recursive-descent arithmetic into pure evaluation lambdas. Multiplication binds before addition; exponentiation is right-associative and binds inside unary minus. Only supported variables/functions are accepted.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `evaluate(...)`: Evaluates arithmetic with the supplied x and optional a/b/c parameters.
- `compile(...)`: Compiles validated arithmetic source into a pure expression evaluator.

9.16.4 FontManager

Source: `util/FontManager.java`

Shared cache of immutable fonts. Caches immutable Font objects by family, style and fractional size. Java logical families supply platform fallback; appearance can vary if installed physical fonts differ.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `get(...)`: Returns a cached immutable font for family, style and positive fractional size.

9.17 Package video (6 files)

9.17.1 AudioCue

Source: `video/AudioCue.java`

Legacy audio metadata holding a source path, project-global start time and volume. ProjectParser converts it into the audio timeline used by the production mixer. The standalone legacy exporter remains silent.

XML connection: legacy root audio.

Authoring note: Legacy root audio is converted into AudioTimeline tracks by ProjectParser. ProductionBuilder mixes it; the source comment claiming all exports are silent is outdated.

Main functions: The type supplies the data, contract or internal operations described above; it is not a separate screenplay command.

9.17.2 RenderContext

Source: `video/RenderContext.java`

Per-frame values; time is local to the active scene. Carries output settings, active scene palette and the scene-local sample time into each object. Camera transforms are copied both on construction and access to prevent shared mutable state.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `cameraTransform(...)`: Returns camera transform for this configured component.

9.17.3 Theme

Source: `video/Theme.java`

Palette and typography shared by objects. Preserves the original seven-component palette constructor and adds semantic derived colours. Named palettes coordinate accents rather than recolouring every item identically. Colours and font must be non-null.

XML connection: theme; scene theme and background style.

Main functions:

- `named(...)`: Resolves a supported named palette; unknown names are rejected.
- `backgroundSecondary(...)`: Derives a softly tinted secondary background from the primary accent.
- `panelBorder(...)`: Derives the coordinated lower-contrast panel outline colour.
- `highlight(...)`: Returns the semantic high-importance colour.
- `axis(...)`: Derives a readable axis colour from foreground and background.
- `glow(...)`: Returns the primary illumination colour.
- `dark(...)`: Returns the original default palette for legacy lessons.
- `panel(...)`: Background-adjacent panel color derived from this palette.
- `muted(...)`: Lower-contrast foreground for indices, line numbers and comments.
- `grid(...)`: Foreground-colored grid lines with explicit alpha.
- `selection(...)`: Translucent primary accent for highlighted code lines.

9.17.4 VideoProject

Source: `video/VideoProject.java`

Shared Java/XML model; do not modify during a render. Holds output settings, default theme, ordered scenes, legacy audio cues and the production audio timeline. Collection accessors return snapshots. Validate once after construction and do not mutate the model during export.

XML connection: edu-video root.

Main functions:

- `audioTimeline(...)`: Returns the separate narration/music/effects timeline.
- `add(...)`: Appends a scene in project playback order.
- `audio(...)`: Configures audio for this component.
- `validate(...)`: Checks scene identity, each scene model and the project frame count.

9.17.5 VideoRenderer

Source: `video/VideoRenderer.java`

Deterministic frame renderer. No UI or real-time clock dependency. Converts a global frame index to a scene-local clock and renders through Java2D. Incoming transitions combine independently rendered frames; no previous-frame simulation state is retained. A renderer owns bounded scene-backdrop caches and should be used by one rendering thread.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `renderFrame(...)`: Renders one in-range frame independently of all previously requested frames.
- `renderTime(...)`: Renders an exact local time; multipart boundaries need not fall on the source frame grid.
- `quality(...)`: Applies consistent antialiasing, fractional metrics and high-quality image rendering hints.
- `renderAllFrames(...)`: Streams numbered PNG files into a directory without an existing frame sequence.

9.17.6 VideoSettings

Source: `video/VideoSettings.java`

Output dimensions and fixed sampling rate. Maps each nonnegative frame index to frame/fps seconds. The frame count rounds duration upward with a small floating-point tolerance and rejects counts exceeding integer range.

XML connection: root width, height and fps.

Main functions:

- `timeAt(...)`: Converts a nonnegative frame index to seconds at the configured frame rate.

9.18 Package xml (11 files)

9.18.1 AnimationParser

Source: `xml/AnimationParser.java`

Scene-local animation syntax. Resolves event targets after all scene objects exist, then builds ordinary timeline tracks or semantic sub-object events. Matrix indices and code selection bounds are validated by their target objects.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `recognizes(...)`: Reports whether an element name is a supported scene event.
- `parse(...)`: Resolves a target and appends validated scene-local tracks or semantic events.

- `value(...)`: Parses a numeric property value or encodes a colour as ARGB.

9.18.2 LocatedXml

Source: `xml/LocatedXml.java`

Hardened SAX loader with DTD/entity denial and retained line numbers. Disallows DTDs, external general/parameter entities and XInclude. A SAX handler preserves source line numbers so semantic validation can identify the originating element.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `read(...)`: Loads a file or `InputStream` with a hardened SAX parser, rejecting DTDs/external entities and retaining source line numbers.

9.18.3 ObjectParser

Source: `xml/ObjectParser.java`

Extensible XML object factory registry shared by the project parser. Registers tag factories and their permitted attributes, applies shared properties, and recursively builds groups. Factories receive the lesson directory and active theme; errors retain XML source context.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `create(...)`: Constructs a scene object from a located element, asset directory and palette.
- `register(...)`: Registers an object factory and the tag-specific permitted XML attributes.
- `parse(...)`: Constructs and validates a scene object, common properties and nested children.

9.18.4 ProductionEvents

Source: `xml/ProductionEvents.java`

Strict screenplay extensions compiled into the existing scene model. Marker references are resolved before normal parsing, so ordinary animation tags can share the same timing markers.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `recognizes(...)`: Recognises screenplay events such as speak, transfer and camera-focus.
- `markers(...)`: Collects scene markers and resolves timing references before ordinary parsing.
- `install(...)`: Registers vectors, statistics, venn, token-equation and calculus.
- `parse(...)`: Validates and compiles a production event into scene objects, tracks or audio cues.

9.18.5 ProductionParser

Source: `xml/ProductionParser.java`

Secure manifest frontend. Lessons remain independently parsed by the existing `ProjectParser`.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `isProduction(...)`: Identifies a manifest using the hardened XML loader, not filename heuristics.
- `parse(...)`: Reads a manifest and resolves all paths relative to its directory.

9.18.6 ProductionWriter

Source: `xml/ProductionWriter.java`

Serializes the small manifest vocabulary; paths are relative to the destination where possible.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `write(...)`: Writes a manifest from ordered parts while retaining global music and output settings.

9.18.7 ProjectFormatException

Source: `xml/ProjectFormatException.java`

Human-readable source location for invalid lesson input. An `IllegalArgumentException` carrying human-readable XML location and validation details. Optional causes preserve parser/I/O diagnostics for callers and preview dialogs.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions: The type supplies the data, contract or internal operations described above; it is not a separate screenplay command.

9.18.8 ProjectParser

Source: `xml/ProjectParser.java`

Versioned XML frontend which builds the public Java model. Parses version 1.0 XML securely, resolves relative assets, selects project/scene themes and validates the completed shared model. Object registration remains injectable for application-specific vocabulary.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `parse(...)`: Parses and validates version 1.0 XML, resolving asset paths relative to its directory.

9.18.9 StandardObjects

Source: `xml/StandardObjects.java`

Built-in XML vocabulary. Add application-specific tags through `ObjectParser.register`. Installs the legacy mathematics, algorithm and primitive vocabulary and delegates new visual components to `VisualObjects`. Registration keeps Java model construction separate from XML traversal.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `install(...)`: Registers built-in maths, algorithm and geometry object factories and their allowed attributes.
- `points(...)`: Parses whitespace-separated x,y pairs and checks finite coordinates.

9.18.10 VisualObjects

Source: `xml/VisualObjects.java`

Registers semantic presentation components without changing the legacy object vocabulary. Registers panels, steps, results, plot annotations, curved arrows and triangles with strict attribute sets. These construct normal scene objects; no alternate renderer is introduced.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `install(...)`: Registers panel, step, result, plot-point, area, curved-arrow and triangle factories.
- `panel(...)`: Applies padding and validated surface settings to a panel.

9.18.11 XmlElement

Source: `xml/XmlElement.java`

Located XML node used by pluggable object factories. Stores an immutable attribute map alongside parse-time child/text buffers. Typed accessors validate finite numbers and strict booleans; `error()` includes line, element and optional id.

XML connection: Supporting Java/application code; no direct object tag with this class name.

Main functions:

- `get(...)`: Returns an attribute string or the supplied fallback.
- `required(...)`: Returns a nonblank attribute or throws a located validation error.
- `number(...)`: Parses a finite numeric attribute, using its fallback when absent.
- `integer(...)`: Parses an exactly integral in-range attribute value.
- `bool(...)`: Parses a strict true/false attribute, using its fallback when absent.
- `only(...)`: Rejects attributes outside the explicitly permitted vocabulary.
- `error(...)`: Creates an exception identifying source line, element name and optional ID.

10 Source coverage and verification

The inventory contains 129 Java files in 18 packages. All are represented above. Four XML templates under `src/main/resources/templates/` were also inspected. The CSS resource supplies the desktop theme, not lesson XML vocabulary. No Java test source files or build descriptor were present in the supplied archive.

The XML vocabulary in the authoring chapters was traced to the parser source. Example snippets were checked statically for element/attribute names, nesting and references; a full Java parse, video render and speech synthesis were not run here. Source-file comments were treated as explanatory material rather than overriding the actual parser or production code.

Archive SHA-256: `62b90b6d78c066136068b6eae0b61c5c31373aeb60308ddfef9be1b9e0dd3c4`.